

Reviewer Pack — Matching Polynomial Mahler Measure Lower-Bounds Tseitin DPLL...

Entry 7351c55abee7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 22:10:02 UTC

Matching Polynomial Mahler Measure Lower-Bounds Tseitin DPLL Tree Size

Entry ID: 7351c55abee7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-16 22:10:02 UTC

1. Conjecture

Field A (mathematical branch): Matching polynomial root theory of Heilmann-Lieb-Godsil (algebraic graph theory: real-rootedness, Mahler measure of μ_G) **Field B** (complexity object): DPLL refutation tree size for Tseitin formulas on random cubic graphs

Statement:

For every connected 3-regular graph G on n vertices with odd-sum $\mathbb{Z}_2$ -charge c (so $\text{Tau}(G,c)$ is unsatisfiable), let $\mu_G(x) = \sum_k (-1)^k m_k(G) x^{n-2k}$ be the matching polynomial of G and let $M(\mu_G) = \prod_{\{r: \mu_G(r)=0, |r| \geq 1\}} |r|$ be its Mahler measure. We conjecture that for every such (G,c) the median DPLL refutation tree size $T(\text{Tau}(G,c))$ over 30 random VSIDS-free variable orderings satisfies $\log_2 T(\text{Tau}(G,c)) \geq \log_2 M(\mu_G) - 1$, equivalently $T(\text{Tau}(G,c)) \geq M(\mu_G)/2$ for every odd-charge cubic Tseitin instance. A single (G,c) with $T < M(\mu_G)/2$ refutes the conjecture.

Rationale (proposer's reasoning):

Heilmann-Lieb constrains roots of μ_G to $[-2\sqrt{d-1}, 2\sqrt{d-1}]$ with multiplicities at the spectral edge controlled precisely by how 'tree-like' G fails to be, and Godsil's covering-tree identity makes $M(\mu_G)$ a quantitative measure of how many cycles obstruct a tree embedding. Since the Ben-Sasson-Wigderson width bound on Tseitin scales with edge expansion — which is exactly what forces matching-polynomial root mass near the spectral edge — the analytic complexity captured by Mahler measure should track DPLL depth. The matching polynomial has been deployed in statistical mechanics and zero-free regions of partition functions, but to our knowledge never as a Tseitin lower bound, giving a genuinely new bridge.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e7fe03b33487ab80

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 odd-charge cubic Tseitin instances (30 random 3-regular graphs for each n in $\{8,10,12,14,16\}$), with median DPLL nodes T taken over 10 fixed pseudorandom variable orderings, support requires $\log_2 T(\text{Tau}(G,c)) \geq \log_2 M(\mu_G) - 1$ on $\geq 95\%$ of instances; any single instance with $T < M(\mu_G)/2$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.93	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas cubic graphs DPLL resolution lower bound matching polynomial - Mahler measure matching polynomial Heilmann Lieb expander Tseitin - matching polynomial roots 3-regular graph proof complexity resolution width

Top relevant hits considered: - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/quant-ph/0703195v3>] Efficient Quantum Algorithm for Hidden Quadratic and Cubic Polynomial Function Graphs - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/1806.00355v1>] Mahler's work on Diophantine equations and subsequent developments - [<http://arxiv.org/abs/math-ph/0003039v1>] Lieb-Thirring Inequalities - [<http://arxiv.org/abs/1806.00356v1>] Mahler's work on the geometry of numbers - [<http://arxiv.org/abs/1906.04920v2>] Polynomial root clustering and explicit deflation - [<http://arxiv.org/abs/1601.01843v2>] Derivatives and real roots of graph polynomials

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=78.7s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_3_regular_graph(n):
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graphs")
    edges = []
    vertices = list(range(n))
    random.shuffle(vertices)
    for i in range(0, n, 2):
        edges.append((vertices[i], vertices[i+1]))
    remaining_vertices = list(range(n))
```

```

while remaining_vertices:
    u = remaining_vertices.pop()
    neighbors = [v for v in vertices if (u, v) in edges or (v, u) in edges]
    if len(neighbors) >= 3:
        continue
    candidates = [v for v in vertices if v != u and v not in neighbors]
    if not candidates:
        continue
    v = random.choice(candidates)
    edges.append((u, v))
return edges

def is_odd_sum_charge(G, c):
    n = len(set(itertools.chain.from_iterable(G)))
    return sum(c[v] for v in range(n)) % 2 != 0

def generate_odd_sum_charge(n):
    c = [random.randint(0, 1) for _ in range(n)]
    while sum(c) % 2 == 0:
        c[random.randint(0, n-1)] ^= 1
    return c

def matching_polynomial(G):
    n = len(set(itertools.chain.from_iterable(G)))
    mu = [0] * (n + 1)
    mu[n] = 1
    for k in range(n, 0, -1):
        for edge in G:
            u, v = edge
            if mu[k] != 0:
                mu[k-2] += mu[k]
    return mu

def mahler_measure(mu):
    roots = []
    for k in range(len(mu)):
        if mu[k] != 0:
            roots.extend([1] * mu[k])
    M = 1.0
    for r in roots:
        if abs(r) >= 1:
            M *= abs(r)
    return M

def generate_cnf(G, c):
    n = len(set(itertools.chain.from_iterable(G)))
    clauses = []
    for edge in G:
        u, v = edge
        clauses.append([u, v])
        clauses.append([-u, -v])
    for v in range(n):
        if c[v] == 1:
            clauses.append([v])
        else:
            clauses.append([-v])

```

```

    return clauses

def dpll(clauses, assignment, order):
    if not clauses:
        return 1
    for clause in clauses:
        if all(lit == 0 for lit in clause):
            return 0
    if not order:
        return 1
    var = order.pop(0)
    for val in [1, -1]:
        new_assignment = assignment.copy()
        new_assignment[var] = val
        new_clauses = []
        for clause in clauses:
            new_clause = []
            for lit in clause:
                if lit == var * val:
                    new_clause = []
                    break
                elif lit != -var * val:
                    new_clause.append(lit)
            if new_clause:
                new_clauses.append(new_clause)
        result = dpll(new_clauses, new_assignment, order.copy())
        if result != 0:
            return result + 1
    return 0

def run_trial(seed):
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(30):
            G = generate_3_regular_graph(n)
            c = generate_odd_sum_charge(n)
            mu_G = matching_polynomial(G)
            M = mahler_measure(mu_G)
            clauses = generate_cnf(G, c)
            order = list(range(n))
            random.shuffle(order)
            T = dpll(clauses, {}, order.copy())
            metric_value = math.log2(T) if T > 0 else 0
            metric_values.append(metric_value)
            instances_tested += 1
            if metric_value < math.log2(M) - 1:
                conjecture_holds = False
                counterexample = f"n={n}, T={T}, M={M}"
                break
    if not conjecture_holds:

```

```

        break

    if not metric_values:
        return {
            "metric_name": "log2_T_minus_log2_M",
            "metric_value": 0,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "No valid instances generated"
        }

    mean_metric = sum(metric_values) / len(metric_values)
    return {
        "metric_name": "log2_T_minus_log2_M",
        "metric_value": mean_metric,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = seeds[next(i for i, r in enumerate(results) if not r["conjecture_holds"])]
        counterexample = results[next(i for i, r in enumerate(results) if not r["conjecture_holds"])]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=0")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test exited with returncode -9 (killed, likely OOM or timeout) and produced no RESULT line or data, so neither the support nor falsification condition can be evaluated.
| next: Re-run with reduced n range (e.g. {8,10,12}) and fewer seeds, and add memory/time limits plus checkpointing so partial per-instance (T, M(μ_G)) data is persisted before any crash.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	115231
2	preregistration	claude_max	opus	0	0	5527
3	novelty	claude_max	opus	0	0	6481
4	novelty	claude_max	opus	0	0	11219
5	test_gen	mistral	codestral-latest	0	0	309845
6	test_gen	mistral	codestral-latest	0	0	310806
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	290787
8	test_gen	mistral	codestral-latest	0	0	310951
9	judge	claude_max	opus	0	0	11288

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1372134 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/7351c55abee7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7351c55abee7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7351c55abee7.tar.gz` (if generated)